

Proposition de projets – Conception robotique

▪ Niveau Explorateur : Découverte & initiation

Les projets Wido :

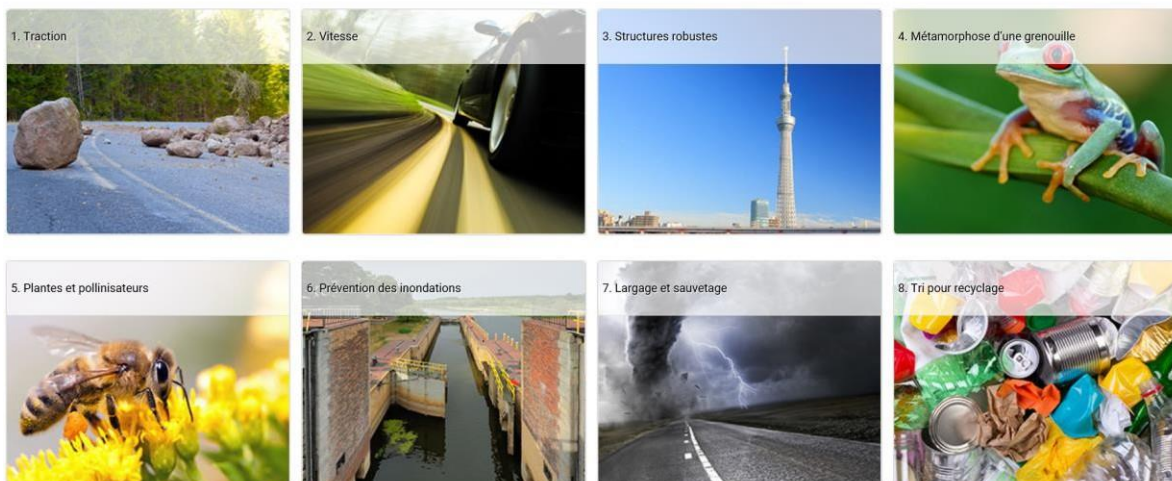
Pour les enfants de 6 à 12 ans

- Lors des **premières séances**, les enfants découvriront et comprendront les notions essentielles grâce aux **projets ludiques Wido**. Ces activités sont conçues pour éveiller leur curiosité, favoriser l'expérimentation et stimuler leur créativité.
- Ensuite, ils mettront en pratique leurs nouvelles compétences sur notre plateforme interactive **app.decli.tech**, où ils pourront créer, tester et s'amuser tout en apprenant.
- **N.B. : Les projets sont déjà préparés sur la plateforme.**

Liste des projets :

Projet découverte

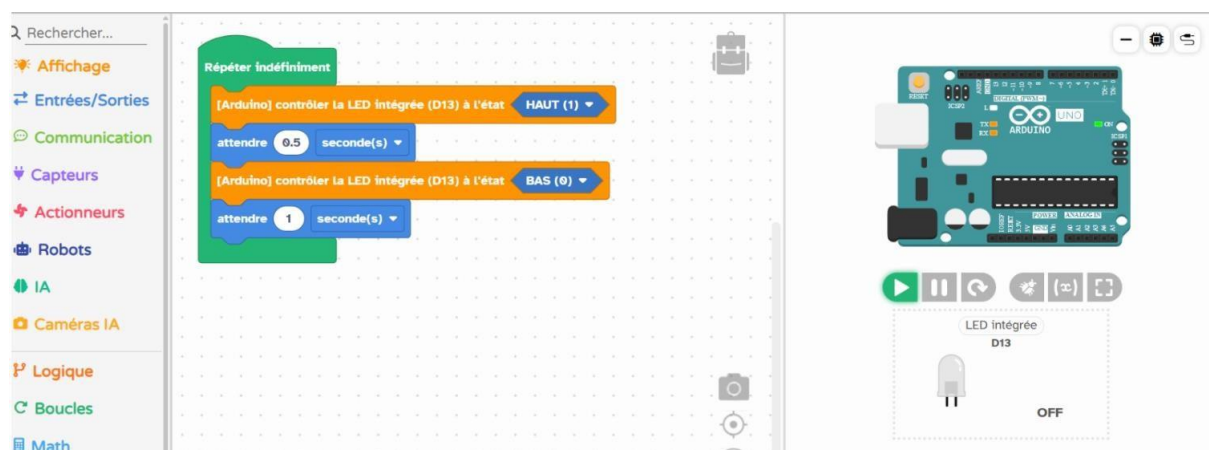




Les projets sur la plateforme app.decli.tech :

Projet 1 : Clignotement de la LED

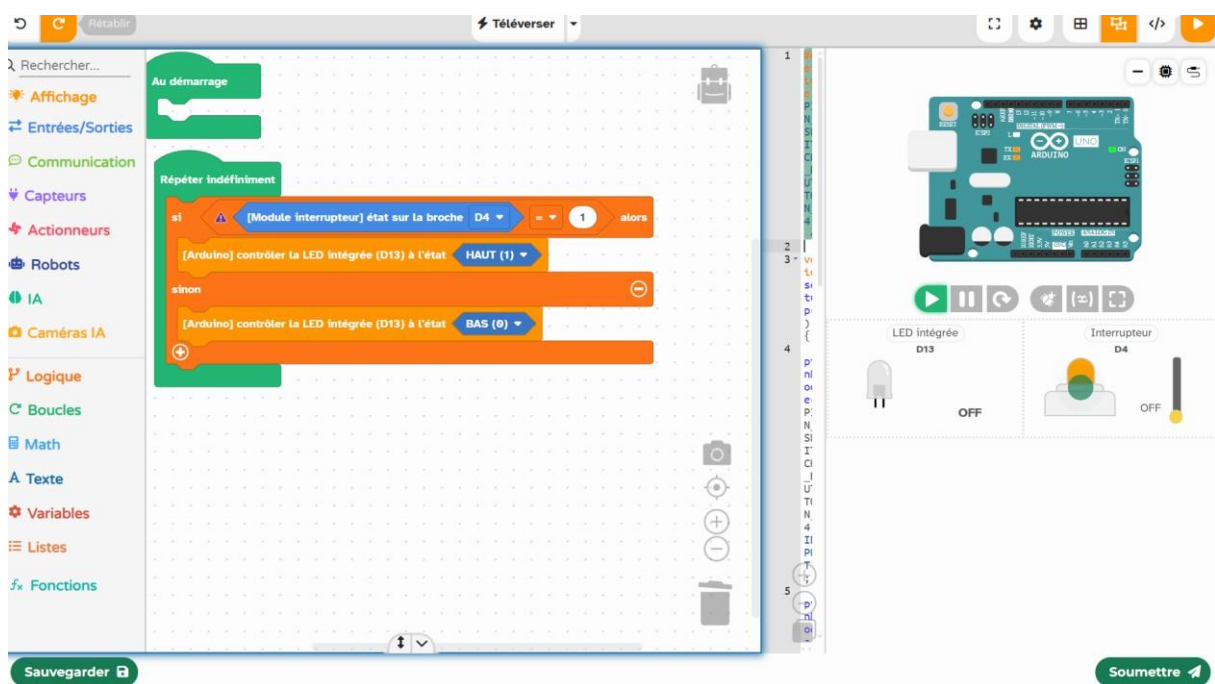
- **Compétences** : Sortie numérique, temporisation, boucle infinie
- **Composants** : LED intégrée à la carte Arduino (broche 13)
- **Description** : La LED intégrée à la carte Arduino s'allume pendant 0,5 seconde puis s'éteint pendant 1 seconde, de façon répétée. Ce projet introduit l'utilisation d'une sortie numérique avec des délais pour créer un clignotement régulier.



🕒 Nombre de séances estimé : de 1 à 2, selon l'âge de l'élève

Projet 2 : Allumage d'une LED avec un bouton-poussoir

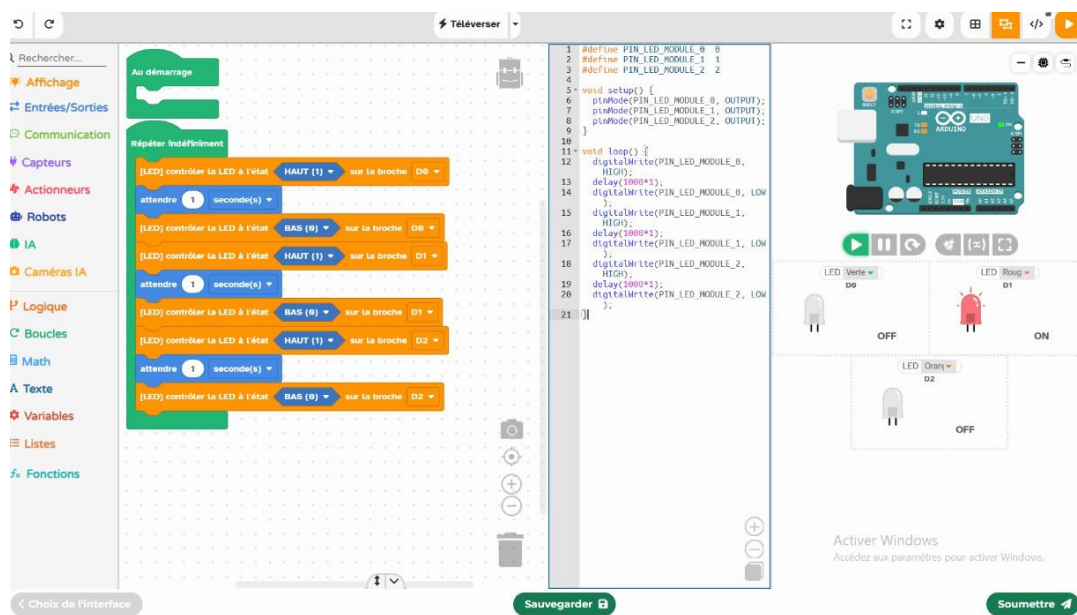
- **Compétences** : Lecture d'entrée numérique, condition simple
- **Composants** : Bouton-poussoir, LED
- **Description** : Lorsqu'un élève appuie sur un bouton, une LED connectée à la broche 13 s'allume. Elle s'éteint dès que le bouton est relâché. Ce projet introduit le contrôle d'actions simples à partir d'une entrée numérique.



🕒 Nombre de séances estimé : de 1 à 2, selon l'âge de l'élève

Projet 3 : Feu tricolore automatique

- **Compétences** : Séquençage, temporisation
- **Composants** : 3 LED (rouge, orange, verte)
- **Description** : Simuler un feu rouge/orange/vert avec délais programmés.

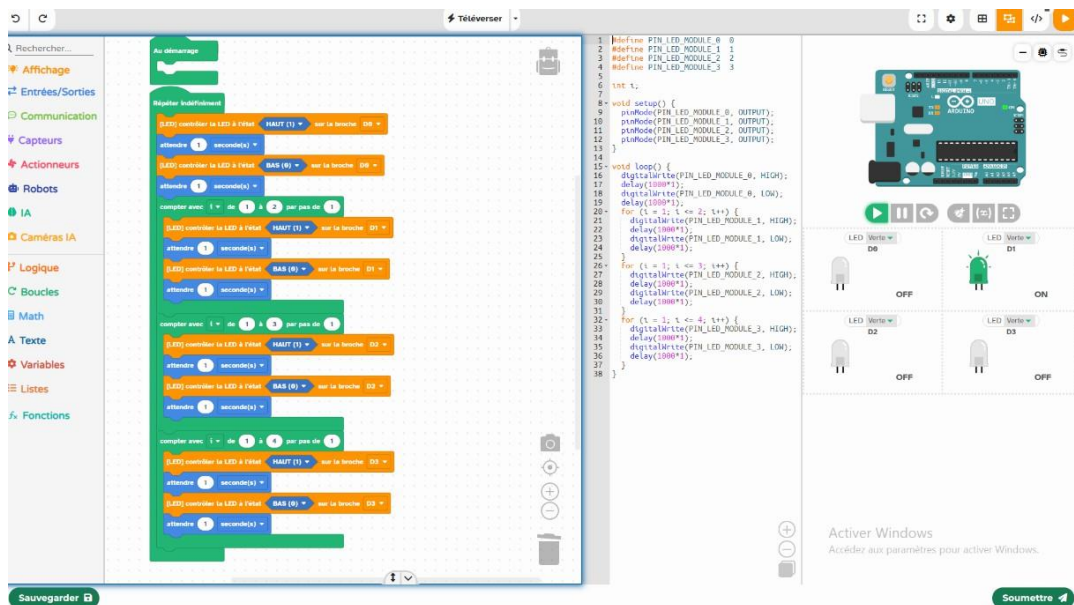


🕒 Nombre de séances estimé : de 1 à 2, selon l'âge de l'élève

Projet 4 : Séquence de clignotements croissants

- **Compétences** : Boucles et logique de programmation séquentielle
- **Composants** : 4 LED, résistances, Arduino
- **Description** : Quatre LED s'allument l'une après l'autre selon une séquence :

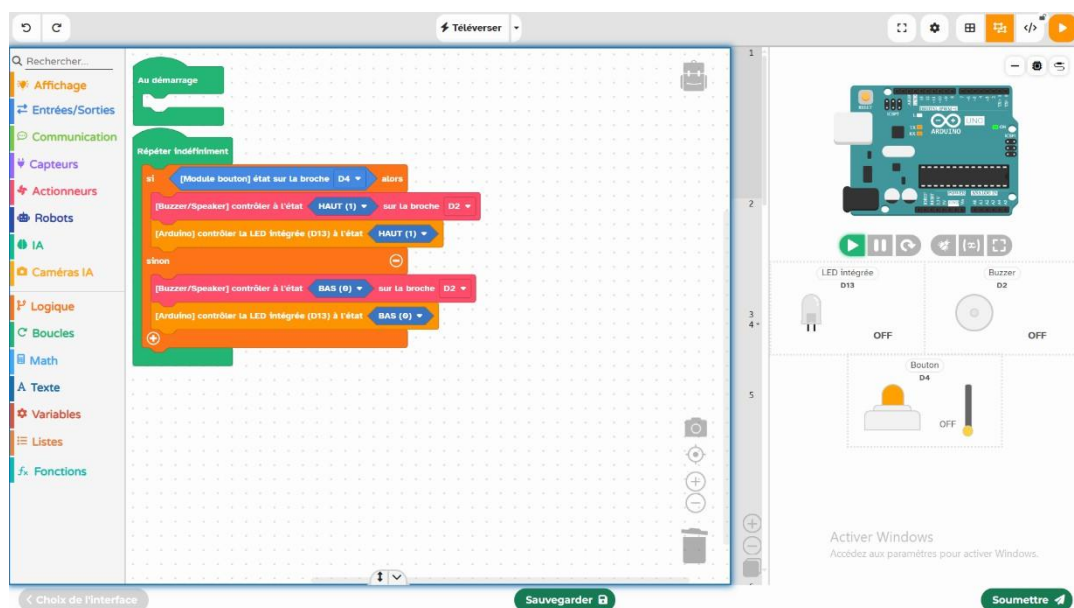
- LED 1 clignote 1 fois
- LED 2 clignote 2 fois
- LED 3 clignote 3 fois
- LED 4 clignote 4 fois



2 Nombre de séances estimé : de 1 à 3, selon l'âge de l'élève

Projet 5 : Alarme sonore avec bouton poussoir

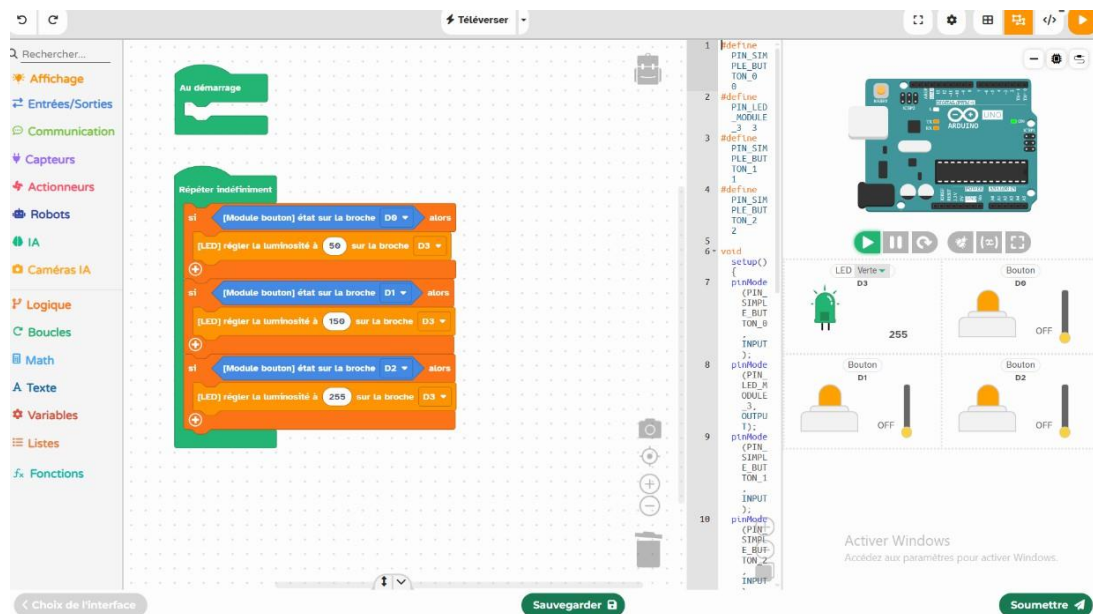
- **Compétences** : Programmation conditionnelle de base
- **Composants** : Bouton poussoir, buzzer, LED
- **Description** : Lorsque l'utilisateur appuie sur un bouton, une LED clignote et un buzzer sonne.



Z Nombre de séances estimé : de 1 à 2, selon l'âge de l'élève

Projet 6 : Contrôle de la luminosité avec 3 boutons

- **Compétences** : Utilisation de sorties PWM, programmation conditionnelle, lecture d'entrées numériques
- **Composants** : 3 boutons poussoirs, 1 LED, Arduino
- **Description** : Trois boutons permettent de contrôler l'intensité lumineuse d'une LED :
 - **Bouton 1** : luminosité faible
 - **Bouton 2** : luminosité moyenne
 - **Bouton 3** : luminosité maximale



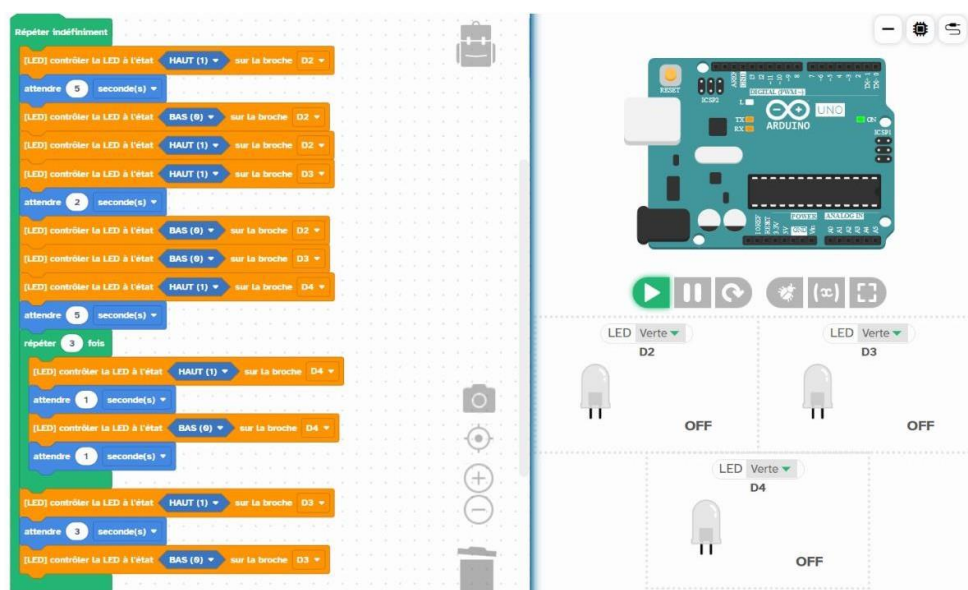
🕒 Nombre de séances estimé : de 1 à 3, selon l'âge de l'élève

Projet 7 : Séquence d'allumage et d'extinction de plusieurs LED

- **Compétences** : Sorties numériques multiples, temporisations, séquences programmées, boucle infinie
- **Composants** : LED sur broches D2, D3, D4
- **Description** :

Le programme répète indéfiniment une séquence d'allumage et d'extinction de trois LED.

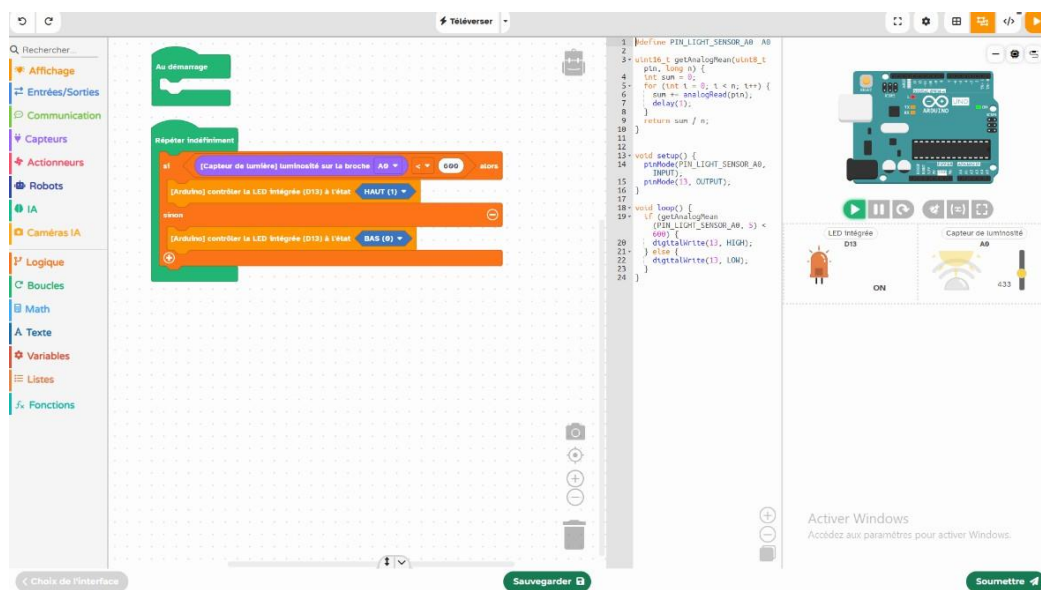
1. La LED connectée à la broche **D2** s'allume pendant 5 secondes, puis s'éteint.
2. Immédiatement après, les LED **D2** et **D3** s'allument ensemble pendant 2 secondes, puis s'éteignent.
3. La LED **D4** s'allume pendant 5 secondes.
4. Trois fois de suite, la LED **D4** clignote : 1 seconde allumée, 1 seconde éteinte.
5. La LED **D3** s'allume pendant 3 secondes, puis s'éteint.



Nombre de séances estimé : de 1 à 3, selon l'âge de l'élève

Projet 8 : Détecteur de lumière (veilleuse automatique)

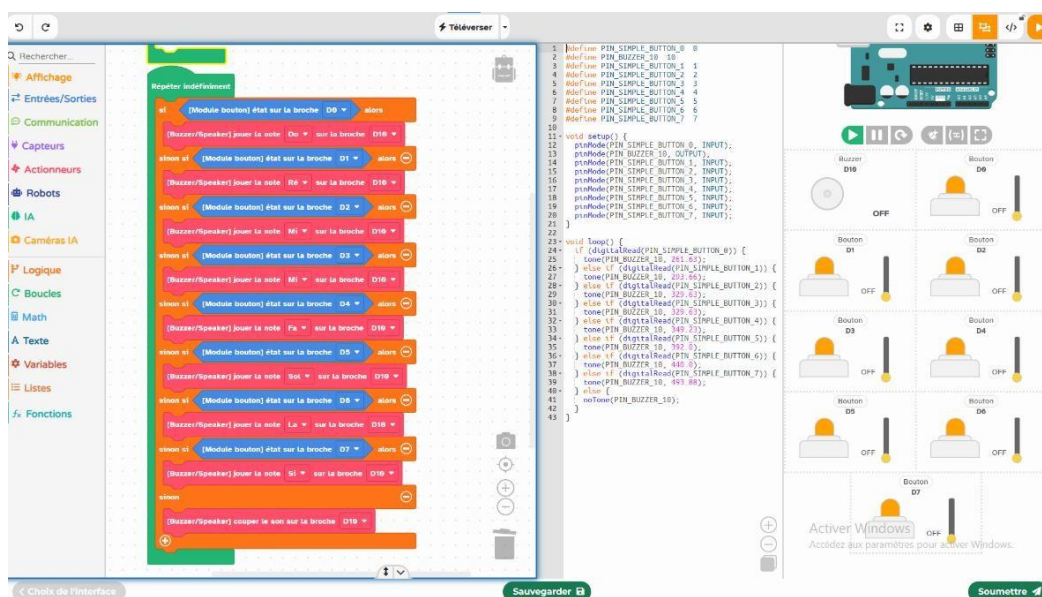
- **Compétences** : Lecture analogique, capteur de luminosité
- **Composants** : Capteur de luminosité, LED, résistance
- **Description** : Une LED s'allume automatiquement si la lumière ambiante est trop faible.



Nombre de séances estimé : de 1 à 2, selon l'âge de l'élève

Projet 9 : Clavier miniature – (mini piano)

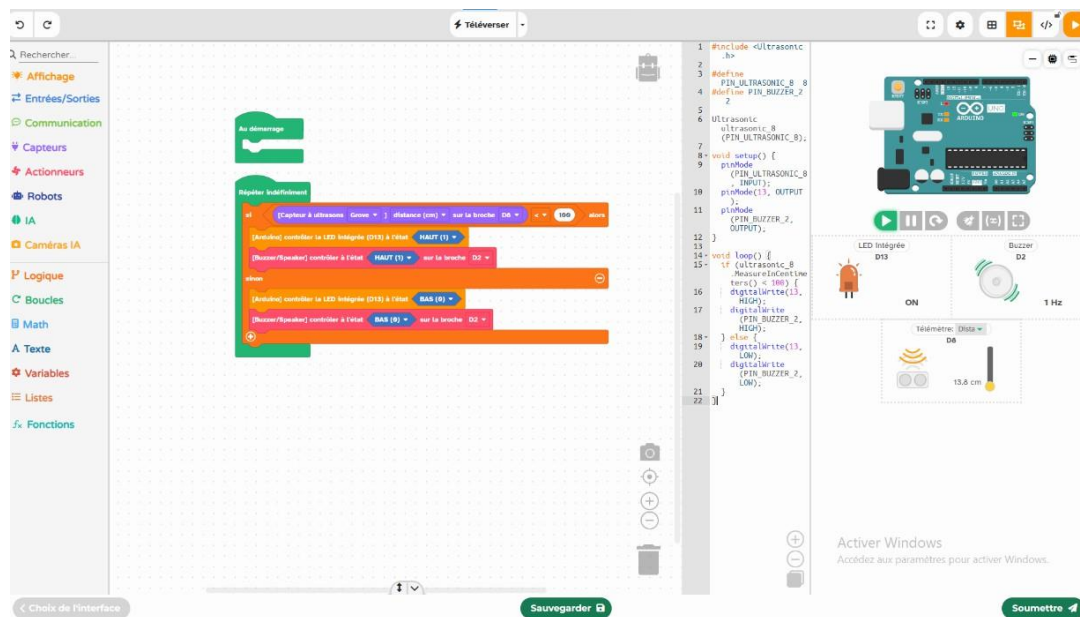
- **Compétences** : Utilisation de plusieurs entrées et gestion des fréquences sonores
- **Composants** : 7 boutons poussoirs, buzzer, Arduino
- **Description** : Chaque bouton joue une note musicale différente (Do, Ré, Mi, Fa, Sol, La, Si) via le buzzer, créant un mini piano interactif.



🕒 Nombre de séances estimé : de 1 à 3, selon l'âge de l'élève

Projet 12 : Détecteur de mouvement

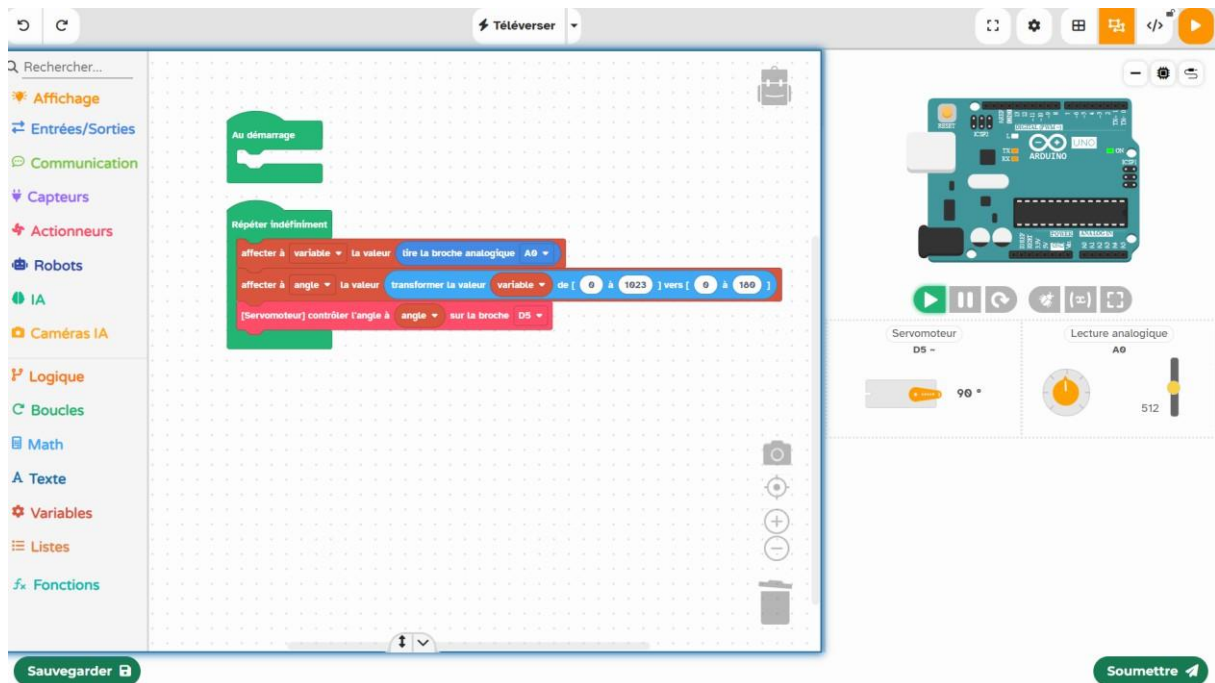
- **Compétences** : Capteur ultrason + logique conditionnelle
- **Composants** : HC-SR04, LED ou buzzer
- **Description** : Détection d'un objet à moins de 100 cm → allumer une alarme.



 Nombre de séances estimé : de 1 à 3, selon l'âge de l'élève

Projet 13 : Alarme lumineuse avec capteur LDR

- **Compétences** : Lecture analogique, conversion de valeurs, contrôle de servomoteur
- **Composants** : Lecteur analogique (ex. potentiomètre), servomoteur
- **Description** : Un lecteur analogique permet de contrôler l'angle d'un servomoteur. La tension mesurée est convertie en angle (de 0 à 180°), et le servomoteur s'oriente en conséquence.



 Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 12 : Détecteur de mouvement

Objectif : Allumer une LED quand le capteur infrarouge détecte un mouvement.

Matériel :

1 capteur **PIR (infra-rouge)**

1 LED

1 résistance (220 Ω)

1 carte Arduino Uno

Fils de connexion

Schéma de câblage

- **Capteur PIR (PIR Motion Sensor)**
 - VCC → 5V sur Arduino
 - GND → GND sur Arduino
 - OUT → D2 sur Arduino (broche numérique)
- **LED**
 - Anode (longue patte) → D13 via résistance 220 Ω
 - Cathode (courte patte) → GND

```
int pirPin = 2;      // Broche du capteur PIR
int ledPin = 13;     // Broche de la LED
int pirState = LOW;  // État initial du PIR
int val = 0;         // Variable pour lire le PIR

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(pirPin, INPUT);
  Serial.begin(9600);
  Serial.println("Capteur de mouvement initialisé...");
}

void loop() {
  val = digitalRead(pirPin); // Lire l'état du PIR

  if (val == HIGH) {         // Mouvement détecté
    digitalWrite(ledPin, HIGH);
    if (pirState == LOW) {
      Serial.println("Mouvement détecté !");
      pirState = HIGH;
    }
  } else {
    digitalWrite(ledPin, LOW);
    if (pirState == HIGH) {
      Serial.println("Pas de mouvement.");
      pirState = LOW;
    }
  }
}
```

Projet 13 : Alarme lumineuse avec capteur LDR

Objectif :

Créer un petit système qui **allume une LED quand il fait sombre** (comme une veilleuse automatique).

🔧 Le capteur **LDR (Light Dependent Resistor)** détecte la quantité de lumière.

Matériel :

- 1 LDR (capteur de lumière)
- 1 résistance (10 k Ω)
- 1 LED + résistance (220 Ω)
- Arduino Uno

```
#define ldrPin A0    // Capteur LDR branché sur l'entrée analogique A0
#define ledPin 13    // LED sur la broche 13

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600); // Pour afficher la valeur dans le moniteur série
}

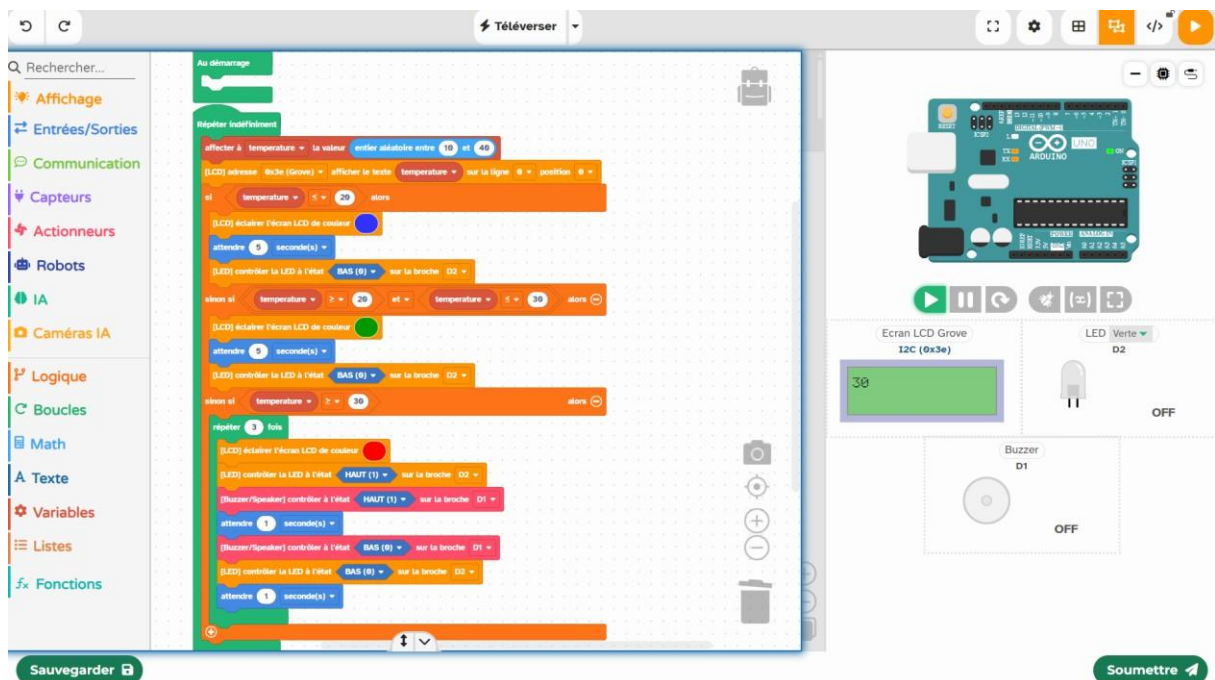
void loop() {
  int valeurLumiere = analogRead(ldrPin); // Lire la valeur de la LDR
  Serial.print("Valeur LDR : ");
  Serial.println(valeurLumiere);

  if (valeurLumiere < 500) {
    // Il fait sombre
    digitalWrite(ledPin, HIGH); // Allumer la LED
  } else {
    // Il fait clair
    digitalWrite(ledPin, LOW);  // Éteindre la LED
  }

  delay(500); // Petite pause pour éviter les lectures trop rapides
}
```


Projet 14 : Affichage de la température et alerte visuelle/sonore

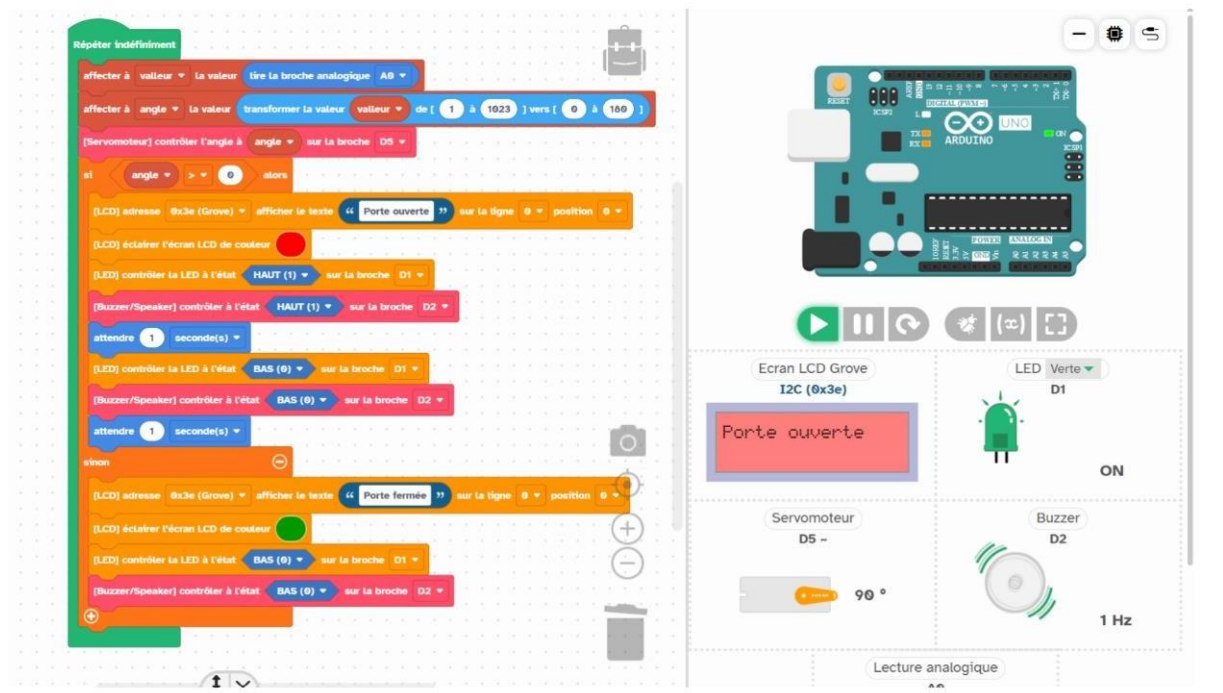
- **Compétences** : Affichage LCD, conditions multiples, signalisation visuelle et sonore
- **Composants** : Écran LCD RGB, LED, buzzer, lecteur de température simulé
- **Description** : Une température aléatoire est simulée et affichée sur un écran LCD.
 - Si la température est basse ($\leq 20^{\circ}\text{C}$), l'écran devient bleu.
 - Si elle est modérée (entre 20°C et 30°C), l'écran devient vert.
 - Si elle est élevée ($\geq 30^{\circ}\text{C}$), l'écran vire au rouge, une LED clignote et un buzzer émet une alerte sonore à plusieurs reprises.



Z Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 15 : Système de sécurité à capteur analogique

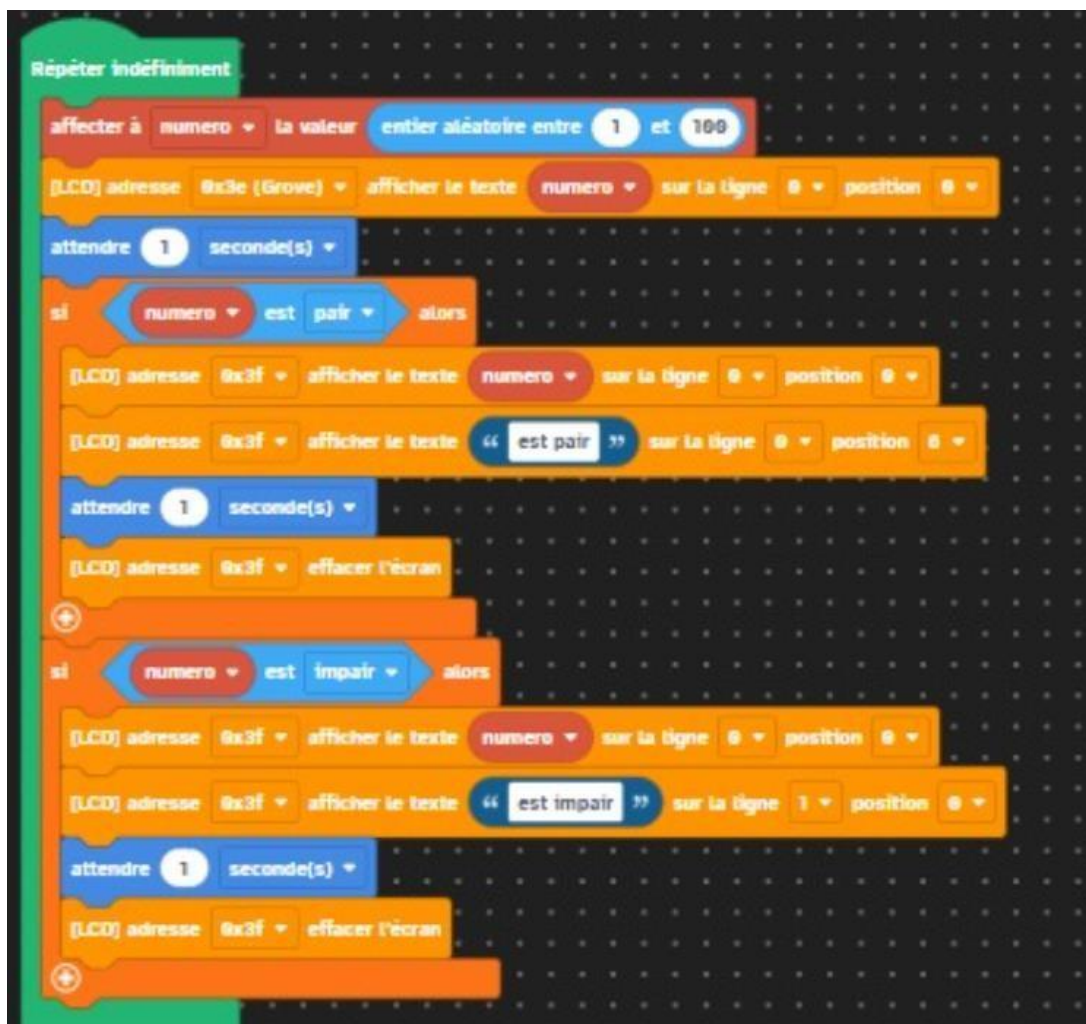
- **Compétences** : Lecture d'entrée analogique, transformation de valeurs, contrôle d'actionneurs multiples
- **Composants** : Capteur analogique (A0), servomoteur (D5), écran LCD , LED verte (D1), buzzer (D2)
- **Description** : Quand la valeur du capteur analogique dépasse un seuil, le servomoteur tourne à 90°, la LED verte s'allume, le buzzer s'éteint, et le LCD affiche "Porte ouverte". Quand la valeur descend sous le seuil, le système inverse tous les états.



🕒 Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 16 : Test de parité avec affichage LCD

- **Compétences** : Génération de nombres aléatoires, opération modulo (%), contrôle d'écran LCD
- **Composants** : Écran LCD Grove (adresse I2C 0x3E)
- **Description** : Le programme génère un nombre aléatoire entre 1 et 100, l'affiche sur l'écran LCD, puis détermine s'il est pair ou impair. Selon le résultat, un message spécifique ("est pair" ou "est impair") s'affiche pendant 1 seconde avant la prochaine itération.



Z Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 17 : Système de vote avec affichage sur écran LCD

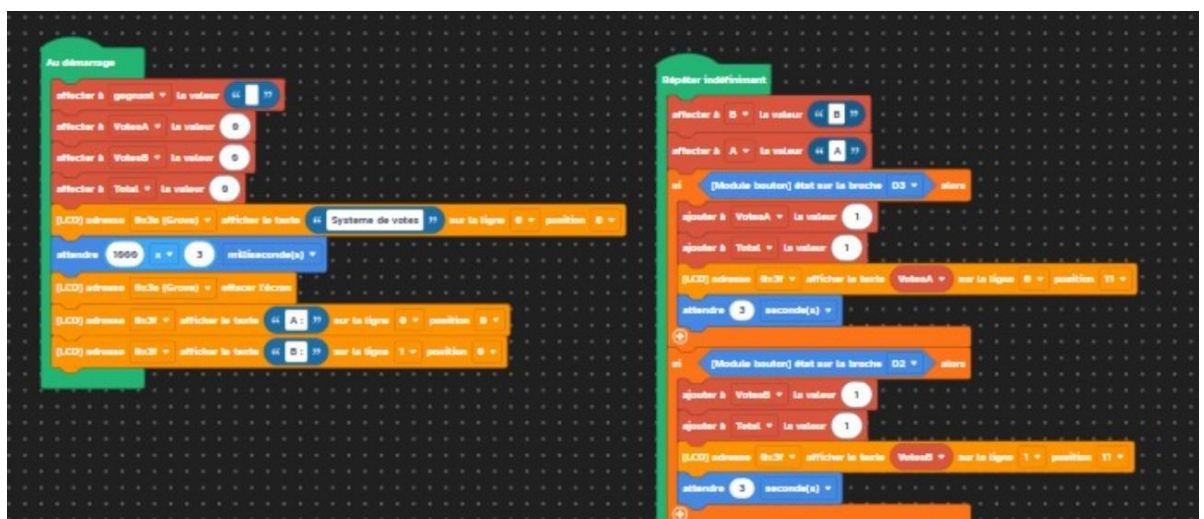
- **Compétences** : Lecture d'entrées numériques (boutons), utilisation d'un écran LCD, stockage et incrémentation de variables, conditions logiques, boucle infinie
- **Composants** :
 - 2 boutons-poussoirs (pour voter pour le candidat A ou B)
 - Écran LCD I2C (adresse 0x3e)
 - Carte Arduino
- **Description** :

Au démarrage, l'écran LCD affiche « Système de votes » pendant une seconde, puis efface l'écran et initialise les compteurs de votes pour les candidats A et B.

Dans une boucle infinie :

 - Si le bouton du **candidat B** (broche D3) est pressé, le compteur de votes pour B est incrémenté de 1, le total général augmente, et l'écran affiche le nouveau score.
 - Si le bouton du **candidat A** (broche D2) est pressé, le compteur de votes pour A est incrémenté de 1, le total général augmente, et l'écran affiche le nouveau score.
 - Le programme compare les scores et affiche sur l'écran LCD le candidat actuellement gagnant ainsi que le nombre total de votes.
 - Si A a plus de votes que B, l'écran affiche « Gagnant : A ».
 - Si B a plus de votes que A, l'écran affiche « Gagnant : B ».
 - En cas d'égalité, l'écran affiche les deux scores à égalité.

Ce projet illustre la **gestion d'événements** via des boutons, la **mise à jour dynamique d'informations sur un écran LCD** et l'utilisation de **conditions pour déterminer un gagnant**.



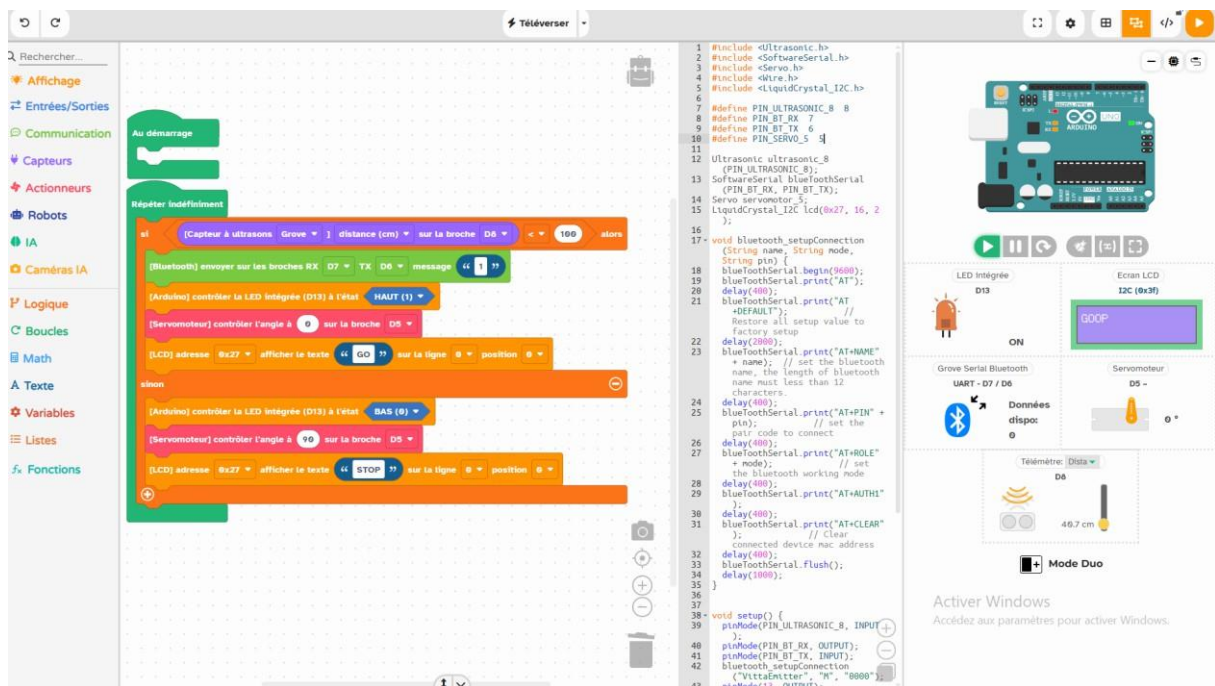


Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 16 : Barrière Bluetooth intelligente

- **Compétences** : Communication sans fil, contrôle de servomoteur, affichage LCD
- **Composants** : Module Bluetooth HC-05, servomoteur, capteur ultrason, écran LCD, Arduino
- **Description** :

Si le capteur ultrason détecte une voiture à moins d'1 mètre, le servomoteur fait tourner la barrière à 90 degrés pour l'ouvrir, une lampe clignotante s'allume, et l'écran LCD affiche « GO ». Sinon, la barrière reste fermée (servomoteur à 0 degré) et l'écran LCD affiche « STOP ».



Nombre de séances estimé : de 1 à 4, selon l'âge de l'élève

Projet 17: Contrôle de LED via la console série

Compétences : Communication série, lecture de caractères, contrôle de sorties numériques

Composants : 4 LED (D2, D3, D4, D5), résistances, Arduino

Description :

L'utilisateur tape un chiffre entre **1 et 4** dans la **console série** pour allumer la LED correspondante. Avant d'allumer une LED, le programme éteint toutes les autres.

Ce projet initie à la **communication série** et au **contrôle d'actions à distance** via la console.

Code exemple :

```
#define LED1 2
#define LED2 3
#define LED3 4
#define LED4 5

void setup() {
    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    pinMode(LED4, OUTPUT);

    Serial.begin(9600);
    Serial.println("Tape 1, 2, 3 ou 4 pour allumer la LED correspondante !");
}

void loop() {
    if (Serial.available() > 0) {
        char input = Serial.read();

        // Éteindre toutes les LED
        digitalWrite(LED1, LOW);
        digitalWrite(LED2, LOW);
        digitalWrite(LED3, LOW);
        digitalWrite(LED4, LOW);

        switch (input) {
            case '1':
                digitalWrite(LED1, HIGH);
                Serial.println("LED 1 allumée !");
                break;
            case '2':
                digitalWrite(LED2, HIGH);
                Serial.println("LED 2 allumée !");
                break;
            case '3':
                digitalWrite(LED3, HIGH);
                Serial.println("LED 3 allumée !");
                break;
            case '4':
                digitalWrite(LED4, HIGH);
                Serial.println("LED 4 allumée !");
                break;
            default:
                Serial.println("Tape un chiffre entre 1 et 4 !");
                break;
        }
    }
}
```


Projet 18 : Comparaison de deux capteurs de lumière HW-488 avec affichage LED et console série

Compétences :

Lecture analogique, comparaison de valeurs, commande de sorties numériques (LED), affichage sur le moniteur série.

Composants :

- 2 capteurs de lumière **HW-488**
- 2 résistances **10 k Ω** (si nécessaire selon le module)
- 2 LED (rouge et verte, par exemple)
- 2 résistances **220 Ω** pour les LED
- Carte **Arduino UNO**
- Fils de connexion
- Câble USB pour la console série

Description :

Deux capteurs **HW-488** sont branchés sur **A0** et **A1**.

Le programme lit la luminosité sur chaque capteur, l'affiche dans la console série et allume la **LED correspondante** :

- Si le **capteur gauche (A0)** détecte plus de lumière → la **LED gauche** s'allume.
- Si le **capteur droit (A1)** reçoit plus de lumière → la **LED droite** s'allume.
- Si la différence est faible → **les deux LED sont éteintes**.

Ce projet illustre la **lecture analogique**, la **comparaison de signaux** et le **pilotage d'actionneurs**.

Schéma de câblage simplifié :

Élément	Broche Arduino	Description
HW-488 gauche	A0	Capteur de lumière gauche
HW-488 droit	A1	Capteur de lumière droit
LED gauche	D2	S'allume si $A0 > A1$
LED droite	D3	S'allume si $A1 > A0$
Toutes les masses	GND	Connexion commune

```
const int capteurGauche = A0;
const int capteurDroit = A1;
const int ledGauche = 2;
const int ledDroite = 3;

int valGauche = 0;
int valDroit = 0;
int seuilDiff = 30; // différence minimale de sensibilité

void setup() {
  pinMode(ledGauche, OUTPUT);
  pinMode(ledDroite, OUTPUT);
}
```

```

Serial.begin(9600);
Serial.println("=== Comparaison de deux capteurs HW-488 ===");
Serial.println("Affichage : A0 | A1 -> Lumière (%) -> LED activée");
Serial.println("-----");
}

void loop() {
  valGauche = analogRead(capteurGauche);
  valDroit = analogRead(capteurDroit);

  int lumG = map(valGauche, 0, 1023, 100, 0); // 100% = sombre, 0% = très lumineux
  int lumD = map(valDroit, 0, 1023, 100, 0);
  int diff = valGauche - valDroit;

  Serial.print("Capteur A0 : ");
  Serial.print(lumG);
  Serial.print("% | Capteur A1 : ");
  Serial.print(lumD);
  Serial.print("% -> ");

  // Éteindre les deux LED avant décision
  digitalWrite(ledGauche, LOW);
  digitalWrite(ledDroite, LOW);

  if (abs(diff) < seuilDiff) {
    Serial.println("Luminosité égale – aucune LED allumée");
  } else if (diff > 0) {
    digitalWrite(ledDroite, HIGH);
    Serial.println("Capteur A1 reçoit PLUS de lumière → LED droite allumée");
  } else {
    digitalWrite(ledGauche, HIGH);
    Serial.println("Capteur A0 reçoit PLUS de lumière → LED gauche allumée");
  }

  Serial.println("-----");
  delay(1000);
}

```

projet 19 : Barrière de Parking Automatique (Servo Standard)

Description : Ce projet simule une barrière de parking. Un capteur ultrasonique (HC-SR04) surveille l'approche d'un véhicule. Si un objet est détecté à moins de 30 cm, un servomoteur *standard* (type SG90, rotation 0-180°) pivote de 90 degrés pour "ouvrir" la barrière. Lorsque l'objet s'éloigne, la barrière se referme.

Composants :

- Arduino Uno
- 1 Capteur ultrasonique HC-SR04
- 1 Servomoteur standard (SG90 ou similaire)
- Breadboard et câbles

Compétences développées :

- Utilisation de la bibliothèque Servo.h
- Contrôle d'un servomoteur en angle (`servo.write(angle)`)
- Lecture de distance (Ultrason)
- Automatisation simple basée sur un seuil de distance.

Code :

```
#include <Servo.h>

// --- Broches ---
#define PIN_SERVO 9
#define PIN_TRIG 10
#define PIN_ECHO 11

// --- Constantes ---
#define DISTANCE_SEUIL 30 // en cm
#define ANGLE_FERME 0
#define ANGLE_OUVERT 90

Servo barriereServo;

void setup() {
  Serial.begin(9600);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  barriereServo.attach(PIN_SERVO);
  barriereServo.write(ANGLE_FERME); // Commencer barrière fermée
}

void loop() {
  long duree;
  int distance;

  // 1. Envoyer l'impulsion (Trig)
  digitalWrite(PIN_TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
```

```
// 2. Lire le retour (Echo)
duree = pulseIn(PIN_ECHO, HIGH);

// 3. Calculer la distance
distance = duree * 0.034 / 2; // (Vitesse du son / 2)

Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");

// 4. Logique de la barrière
if (distance < DISTANCE_SEUIL) {
  barriereServo.write(ANGLE_OUVERT); // Ouvrir la barrière
} else {
  barriereServo.write(ANGLE_FERME); // Fermer la barrière
}

delay(500); // Attendre avant la prochaine mesure
}
```

projet 20 : Feu de Circulation avec Bouton Piéton

Description : Ce projet simule un feu de circulation simple (Rouge, Jaune, Vert) qui fonctionne en boucle. Un bouton poussoir est ajouté pour permettre à un "piéton" de demander le passage. Lorsque le bouton est pressé, le cycle s'interrompt pour passer au rouge (pour les voitures) plus rapidement, permettant au piéton de traverser.

Composants :

- Arduino Uno
- 3 LEDs (1x Rouge, 1x Jaune, 1x Verte)
- 1 Bouton poussoir
- 3 Résistances de 220 Ω (pour les LEDs)
- 1 Résistance de 10 k Ω (pull-down pour le bouton)
- Breadboard et câbles

Compétences développées :

- Programmation séquentielle (`delay()`)
- Gestion des sorties numériques (`digitalWrite()`)
- Lecture d'une entrée numérique (`digitalRead()`)
- Utilisation de la logique conditionnelle (`if`) pour interrompre un cycle.

Code :

```
// --- Broches des LEDs ---
#define LED_ROUGE 10
#define LED_JAUNE 9
#define LED_VERT 8
// --- Broche du bouton ---
#define BOUTON_PIETON 2
void setup() {
  pinMode(LED_ROUGE, OUTPUT);
  pinMode(LED_JAUNE, OUTPUT);
```

```

pinMode(LED_VERTE, OUTPUT);
pinMode(BOUTON_PIETON, INPUT);
// Utilise une résistance de 10kΩ en pull-down sur le bouton
}

void loop() {
  // --- Cycle normal (Voitures) ---
  // 1. Feu Vert
  digitalWrite(LED_VERTE, HIGH);
  digitalWrite(LED_JAUNE, LOW);
  digitalWrite(LED_ROUGE, LOW);
  // Vérifie si le piéton appuie pendant 5 secondes
  checkBouton(5000);
  // 2. Feu Jaune
  digitalWrite(LED_VERTE, LOW);
  digitalWrite(LED_JAUNE, HIGH);
  digitalWrite(LED_ROUGE, LOW);
  delay(2000); // Jaune fixe
  // 3. Feu Rouge
  digitalWrite(LED_VERTE, LOW);
  digitalWrite(LED_JAUNE, LOW);
  digitalWrite(LED_ROUGE, HIGH);
  delay(5000); // Rouge fixe
}

// Fonction pour vérifier le bouton pendant un certain temps
void checkBouton(int duree_ms) {
  long startTime = millis();
  while (millis() - startTime < duree_ms) {
    if (digitalRead(BOUTON_PIETON) == HIGH) {
      // Le piéton a appuyé ! On sort de la boucle
      break;
    }
    delay(10); // Petite pause
  }
}

```

- **Niveau Architecte : Logique & structuration**

Même projet au niveau explorateur, mais avec le code.

Niveau leader : Collaboration & gestion de projet

Project1 : Système d'alarme avec capteur PIR + LED + Buzzer

Objectif :

Créer un système de sécurité qui détecte la présence d'une personne et déclenche une alarme (LED + buzzer).

Matériel :

- 1 capteur PIR (mouvement)
- 1 LED
- 1 buzzer
- Résistance 220 Ω
- Câbles de connexion

```
1  #define pirPin 2      // Capteur PIR branché sur D2
2  #define ledPin 13     // LED sur D13
3  #define buzzerPin 8   // Buzzer sur D8
4
5  void setup() {
6      pinMode(pirPin, INPUT);
7      pinMode(ledPin, OUTPUT);
8      pinMode(buzzerPin, OUTPUT);
9      Serial.begin(9600);
10     Serial.println("Système d'alarme prêt !");
11 }
12
13 void loop() {
14     int mouvement = digitalRead(pirPin);
15     if (mouvement == HIGH) {
16         Serial.println("⚠ Mouvement détecté !");
17         digitalWrite(ledPin, HIGH);
18         digitalWrite(buzzerPin, HIGH);
19         delay(1000);
20     } else {
21         Serial.println("Aucun mouvement.");
22         digitalWrite(ledPin, LOW);
23         digitalWrite(buzzerPin, LOW);
24     }
25     delay(500);
26 }
27
28
29
30
```

Projet 2: Mini-Serre Intelligente

Description :

Ce projet permet de surveiller automatiquement la **luminosité** et l'**humidité** d'une mini-serre à l'aide d'un **Arduino Uno**.

Les valeurs mesurées par le **capteur DHT11** (humidité) et la **LDR** (lumière) sont affichées sur un **écran LCD I2C**.

Si la lumière est faible, une **LED** s'allume pour éclairer la serre ; si l'humidité est basse, une autre **LED** signale le besoin d'arrosage.

Composants :

- Arduino Uno
- Capteur DHT11
- Capteur de lumière (LDR)
- Écran LCD I2C
- 2 LED + résistances
- Breadboard et câbles

Compétences développées :

- Programmation Arduino (C/C++)
- Lecture de capteurs analogiques et numériques
- Affichage LCD I2C
- Automatisation simple basée sur des seuils

code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define PIN_LIGHT_SENSOR_A0 A0
#define PIN_DHT11_SENSOR_8 8
#define PIN_LED_MODULE_2 2
#define PIN_LED_MODULE_3 3

LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht11_8(PIN_DHT11_SENSOR_8, DHT11);

int lumiere;
float humidite;

uint16_t getAnalogMean(uint8_t pin, long n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
```

```

    sum += analogRead(pin);
    delay(1);
}
return sum / n;
}

void setup() {
  lcd.init();
  lcd.backlight();
  pinMode(PIN_LIGHT_SENSOR_A0, INPUT);
  pinMode(PIN_DHT11_SENSOR_8, INPUT);
  dht11_8.begin();
  pinMode(PIN_LED_MODULE_2, OUTPUT);
  pinMode(PIN_LED_MODULE_3, OUTPUT);
  lcd.setCursor(0, 0);
  lcd.print(String("Mini-Serre"));
  delay(2000);
  lcd.clear();
}

void loop() {
  lumiere = getAnalogMean(PIN_LIGHT_SENSOR_A0, 5);
  humidite = dht11_8.readHumidity();
  lcd.setCursor(0, 0);
  lcd.print(String("Lum:"));
  lcd.setCursor(5, 0);
  lcd.print(String(String(lumiere)));
  lcd.setCursor(0, 1);
  lcd.print(String("Hum:"));
  lcd.setCursor(5, 1);
  lcd.print(String(String(humidite)));
  if (lumiere < 500) {
    digitalWrite(PIN_LED_MODULE_2, HIGH);
  } else {
    digitalWrite(PIN_LED_MODULE_2, LOW);
  }
  if (humidite < 50) {
    digitalWrite(PIN_LED_MODULE_3, HIGH);
  } else {
    digitalWrite(PIN_LED_MODULE_3, LOW);
  }
  delay(1000);
}

```

nb séance estimée: 1,2,3 selon l'âge

□ **Projet 3: Contrôle de direction avec 4 boutons**

Description :

Ce projet permet de contrôler un **robot à deux roues** à l'aide de **4 boutons poussoirs** :

- ▲ **Avancer**
- ▼ **Reculer**
- ◀ **Gauche**
- ▶ **Droite**

Chaque bouton commande les deux **servomoteurs continus** pour orienter le robot dans la direction correspondante.

Composants :

- Arduino Uno
 - 2 Servomoteurs continus
 - 4 Boutons poussoirs
 - 4 Résistances de 10 k Ω (pull-down)
 - Breadboard et fils de connexion
-

Compétences développées :

- Programmation Arduino (C/C++)
- Contrôle de moteurs via la bibliothèque **Servo.h**
- Utilisation de **boutons d'entrée numérique**
- Commande directionnelle d'un robot simple

```
#include <Servo.h>

// === Définition des broches ===
#define PIN_BTN_AVANCER 2
#define PIN_BTN_RECULER 3
#define PIN_BTN_GAUCHE 4
#define PIN_BTN_DROITE 5

#define SERVO_GAUCHE 9
#define SERVO_DROIT 10

Servo moteurG;
Servo moteurD;

// === Fonctions de mouvement ===
void avancer() {
    moteurG.write(0); // sens avant
    moteurD.write(180); // sens avant
}

void reculer() {
    moteurG.write(180); // sens arrière
    moteurD.write(0); // sens arrière
}

void tourner_gauche() {
    moteurG.write(90); // stop gauche
    moteurD.write(180); // tourne à droite
}
```

```

void tourner_droite() {
  moteurG.write(0); // tourne à gauche
  moteurD.write(90); // stop droite
}

void arret() {
  moteurG.write(90); // arrêt (vitesse neutre)
  moteurD.write(90);
}

void setup() {
  pinMode(PIN_BTN_AVANCER, INPUT);
  pinMode(PIN_BTN_RECULER, INPUT);
  pinMode(PIN_BTN_GAUCHE, INPUT);
  pinMode(PIN_BTN_DROITE, INPUT);

  moteurG.attach(SERVO_GAUCHE);
  moteurD.attach(SERVO_DROIT);

  arret(); // on démarre à l'arrêt
}

void loop() {
  if (digitalRead(PIN_BTN_AVANCER)) {
    avancer();
  }
  else if (digitalRead(PIN_BTN_RECULER)) {
    reculer();
  }
  else if (digitalRead(PIN_BTN_GAUCHE)) {
    tourner_gauche();
  }
  else if (digitalRead(PIN_BTN_DROITE)) {
    tourner_droite();
  }
  else {
    arret();
  }
}

```

🔧 Projet 4: Contrôle de Moteur avec Capteur Ultrasonique

Description :

Ce projet permet de **contrôler automatiquement deux moteurs** à l'aide de **capteurs ultrasoniques**.
Le système mesure la **distance** d'obstacles et adapte le mouvement :

- si un obstacle est **trop proche**, les moteurs s'arrêtent ou changent de direction ;
- sinon, le robot continue d'avancer.

Composants :

- Arduino Uno

- 3 capteurs ultrasoniques HC-SR04
- 2 servomoteurs continus
- Breadboard et câbles

Compétences développées :

- Programmation Arduino avec **Servo.h** et **Ultrasonic.h**
- Lecture de distance avec capteurs ultrasoniques
- Contrôle automatique de moteurs
- Réalisation d'un système de détection et réaction en temps réel

```
#include <Ultrasonic.h>
#include <Servo.h>

#define PIN_ULTRASONIC_3 3
#define PIN_ULTRASONIC_2 2
#define PIN_ULTRASONIC_4 4
#define PIN_CONTINUOUS_SERVO_0 0
#define PIN_CONTINUOUS_SERVO_1 1

Ultrasonic ultrasonic_3(PIN_ULTRASONIC_3);
Ultrasonic ultrasonic_2(PIN_ULTRASONIC_2);
Ultrasonic ultrasonic_4(PIN_ULTRASONIC_4);
Servo servomotor_0;
Servo servomotor_1;

float distance_gauche;
float disatance_droite;
float distance;

void avancer() {
  servomotor_0.write(90*(1-100/100));
  servomotor_1.write(90*(1-100/100));
}

void tournerGauche() {
  servomotor_0.write(90*(1-0/100));
  servomotor_1.write(90*(1-100/100));
}

void tournerdroite() {
  servomotor_1.write(90*(1-0/100));
  servomotor_0.write(90*(1-100/100));
}

void arreter() {
  servomotor_1.write(90*(1-0/100));
```

```

servomotor_0.write(90*(1-0/100));
}

void setup() {
  pinMode(PIN_ULTRASONIC_3, INPUT);
  pinMode(PIN_ULTRASONIC_2, INPUT);
  pinMode(PIN_ULTRASONIC_4, INPUT);
  servomotor_0.attach(PIN_CONTINUOUS_SERVO_0);
  servomotor_1.attach(PIN_CONTINUOUS_SERVO_1);
}

void loop() {
  distance_gauche = ultrasonic_3.MeasureInCentimeters();
  disatance_droite = ultrasonic_2.MeasureInCentimeters();
  distance = ultrasonic_4.MeasureInCentimeters();
  if (disatance_droite < 20) {
    delay(500);
    tournerGauche();
  } else if (distance_gauche < 20) {
    delay(500);
    tournerdroite();
  } else if (distance < 20) {
    arreter();
  } else {
    delay(500);
    avancer();
  }
}

```

Project5 : Robot suiveur de ligne

Objectif :

Construire un robot capable de suivre une ligne noire tracée sur le sol grâce à des capteurs infrarouges.

Matériel :

- 2 capteurs IR (suiveurs de ligne)
- 2 moteurs à courant continu + module L298N
- 1 châssis robot (ou simulation sur Tinkercad)
- Arduino Uno


```
1  #define capteurG 2
2  #define capteurD 3
3  #define moteurG1 8
4  #define moteurG2 9
5  #define moteurD1 10
6  #define moteurD2 11
7
8  void setup() {
9      pinMode(capteurG, INPUT);
10     pinMode(capteurD, INPUT);
11     pinMode(moteurG1, OUTPUT);
12     pinMode(moteurG2, OUTPUT);
13     pinMode(moteurD1, OUTPUT);
14     pinMode(moteurD2, OUTPUT);
15 }
16
17 void loop() {
18     int gauche = digitalRead(capteurG);
19     int droite = digitalRead(capteurD);
20
21     // Si les deux capteurs voient du blanc → avancer
22     if (gauche == HIGH && droite == HIGH) {
23         avancer();
24     }
25     // Si le capteur gauche voit du noir → tourner à gauche
26     else if (gauche == LOW && droite == HIGH) {
27         tournerGauche();
28     }
29     // Si le capteur droit voit du noir → tourner à droite
30     else if (gauche == HIGH && droite == LOW) {
31         tournerDroite();
32     }
```

```

17 void loop() {
34     else {
35         stopRobot();
36     }
37 }
38
39 void avancer() {
40     digitalWrite(moteurG1, HIGH);
41     digitalWrite(moteurG2, LOW);
42     digitalWrite(moteurD1, HIGH);
43     digitalWrite(moteurD2, LOW);
44 }
45
46 void tournerGauche() {
47     digitalWrite(moteurG1, LOW);
48     digitalWrite(moteurG2, LOW);
49     digitalWrite(moteurD1, HIGH);
50     digitalWrite(moteurD2, LOW);
51 }
52
53 void tournerDroite() {
54     digitalWrite(moteurG1, HIGH);
55     digitalWrite(moteurG2, LOW);
56     digitalWrite(moteurD1, LOW);
57     digitalWrite(moteurD2, LOW);
58 }
59
60 void stopRobot() {
61     digitalWrite(moteurG1, LOW);
62     digitalWrite(moteurG2, LOW);
63     digitalWrite(moteurD1, LOW);
64     digitalWrite(moteurD2, LOW);
65 }

```

Project 6 : Serre intelligente automatisée

Compétences :

Lecture de capteurs multiples (humidité, température, lumière), contrôle automatique d'actionneurs (pompe, ventilateur, LED), affichage LCD, programmation de seuils, gestion énergétique, travail collaboratif et suivi de l'environnement.

Composants :

- Carte Arduino UNO + Shield Sensor V5.0
- Capteur d'humidité du sol
- Capteur de température/humidité DHT11
- Capteur de lumière HW-488
- Relais + mini pompe ou LED simulant l'arrosage
- Relais + petit ventilateur ou LED simulant ventilation
- Écran LCD 16x2 I2C
- Buzzer d'alerte

Description :

La serre intelligente mesure l'humidité du sol, la température et la luminosité en continu.

- Si le sol est trop sec → la pompe s'active pour arroser les plantes.

- Si la température est trop élevée → le ventilateur se met en marche pour refroidir l'air.
- Si la luminosité est insuffisante → une LED d'alerte signale que la plante manque de lumière.

Toutes les mesures et les actions des actionneurs sont affichées en temps réel sur le LCD. Un buzzer avertit en cas de conditions critiques (trop sec, trop chaud ou lumière insuffisante).

Nombre de séances estimé : de 12 à 18, selon le niveau des élèves et le nombre de fonctionnalités implémentées.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// ===== Pins =====
#define PIN_SOIL A0          // Capteur humidité sol
#define PIN_LIGHT A1        // Capteur luminosité
#define DHTPIN 2            // DHT11
#define DHTTYPE DHT11

#define PIN_PUMP 7           // Relais pompe
#define PIN_FAN 6           // Relais ventilateur
#define PIN_BUZZER 8        // Buzzer
#define PIN_LED_LIGHT 9     // LED signal faible lumière

// ===== Seuils =====
#define HUMIDITY_THRESHOLD 400 // Sol trop sec
#define TEMP_THRESHOLD 30      // Température trop élevée
#define LIGHT_THRESHOLD 300    // Luminosité trop faible

// ===== Objets =====
DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    Serial.begin(9600);

    pinMode(PIN_PUMP, OUTPUT);
    pinMode(PIN_FAN, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    pinMode(PIN_LED_LIGHT, OUTPUT);

    pinMode(PIN_SOIL, INPUT);
    pinMode(PIN_LIGHT, INPUT);

    dht.begin();
    lcd.init();
    lcd.backlight();

    lcd.setCursor(0,0);
    lcd.print("Serre Intelligente");
    lcd.setCursor(0,1);
    lcd.print("Initialisation...");
    delay(2000);
    lcd.clear();
}

void loop() {
    // ===== Lecture des capteurs =====
    int soilMoisture = analogRead(PIN_SOIL);
    int lightLevel = analogRead(PIN_LIGHT);
    float temperature = dht.readTemperature();
```

```

float humidity = dht.readHumidity();

// ===== Affichage LCD =====
lcd.setCursor(0,0);
lcd.print("HumSol:");
lcd.print(soilMoisture);
lcd.print(" L:");
lcd.print(lightLevel);

lcd.setCursor(0,1);
lcd.print("T:");
lcd.print(temperature);
lcd.print("C H:");
lcd.print(humidity);
lcd.print("%");

// ===== Gestion pompe =====
if (soilMoisture > HUMIDITY_THRESHOLD) {
    digitalWrite(PIN_PUMP, HIGH); // activer pompe
    tone(PIN_BUZZER, 1000);       // alerte sol sec
} else {
    digitalWrite(PIN_PUMP, LOW);
    noTone(PIN_BUZZER);
}

// ===== Gestion ventilateur =====
if (temperature > TEMP_THRESHOLD) {
    digitalWrite(PIN_FAN, HIGH);
    tone(PIN_BUZZER, 1500, 300); // bip court alerte chaleur
} else {
    digitalWrite(PIN_FAN, LOW);
}

// ===== Gestion LED lumière =====
if (lightLevel < LIGHT_THRESHOLD) {
    digitalWrite(PIN_LED_LIGHT, HIGH); // lumière insuffisante
} else {
    digitalWrite(PIN_LED_LIGHT, LOW);
}

delay(1000); // Mise à jour toutes les 1s
}

```

Projet 7 : Robot éviteur d'obstacles intelligent

Compétences :

- Lecture de distance par capteur ultrason HC-SR04
- Contrôle de servomoteur (balayage angulaire)
- Commande moteur via driver L298N
- Logique décisionnelle et choix intelligent de trajectoire
- Travail collaboratif (câblage + mécanique + code)
- Débogage et optimisation du comportement robotique

Composants :

- Châssis robot à **2 roues** + roue libre
- **2 moteurs DC** + support moteur
- Carte **Arduino UNO**
- Driver moteur **L298N**
- Capteur ultrason **HC-SR04**

- **Mini servomoteur SG-90** (montage support)
- Support pour capteur ultrason rotatif
- Support **4 piles AA** + interrupteur
- 10 × fils Dupont M/F
- 10 × fils Dupont F/F
- Câble USB Arduino

Description :

Ce projet consiste à concevoir un **véhicule autonome** capable d'**éviter les obstacles** grâce à une **mesure de distance ultrasonique**.

Le capteur HC-SR04, monté sur un **servomoteur**, balaie l'avant du robot :

1. Le robot avance en ligne droite.
2. Lorsqu'un obstacle est détecté **à moins de 20 cm**, il s'arrête.
3. Le servomoteur effectue un balayage : **droite** → **gauche**.
4. Le robot compare les distances mesurées dans chaque direction.
5. Il se tourne automatiquement vers la direction **la plus sûre**.
6. Il reprend son mouvement rectiligne.

Ce robot simule un comportement **semi-intelligent** en choisissant l'itinéraire optimal sans intervention humaine.

Ce kit est spécialement adapté à l'**apprentissage STEAM**, aux **compétitions robotiques** et aux **défis d'évitement**.

Nombre de séances estimé : de **12 à 24**, selon le niveau et les extensions choisies.

```
#include <Servo.h>

// --- Définition des broches moteurs (L298N) ---
#define ENA 5    // Vitesse moteur gauche (PWM)
#define IN1 6    // Sens moteur gauche
#define IN2 7
#define ENB 10   // Vitesse moteur droit (PWM)
#define IN3 8    // Sens moteur droit
#define IN4 9

// --- Ultrason HC-SR04 ---
#define TRIG 3
#define ECHO 2

// --- Servo ---
Servo servoHead;

// Variables de distance
long duration;
int distance;

// Fonction pour mesurer la distance en cm
int measureDistance() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    duration = pulseIn(ECHO, HIGH);
    return duration * 0.034 / 2; // (vitesse du son)
```

```

}

// --- Moteurs ---
void avancer() {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, 150);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENB, 150);
}

void reculer() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(ENA, 150);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    analogWrite(ENB, 150);
}

void gauche() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(ENA, 150);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
    analogWrite(ENB, 150);
}

void droite() {
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, 150);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
    analogWrite(ENB, 150);
}

void arret() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

// -----

void setup() {
    Serial.begin(9600);

    // Pins moteurs
    pinMode(ENA, OUTPUT);
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(ENB, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    // Ultrason
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
}

```

```

// Servo
servoHead.attach(11);
servoHead.write(90);
delay(500);

Serial.println("Robot eviteur d'obstacles prêt !");
}

void loop() {
    distance = measureDistance();
    Serial.print("Distance devant : ");
    Serial.println(distance);

    if (distance > 20) {
        avancer();
    } else {
        arret();
        delay(300);

        // Scan droite
        servoHead.write(30);
        delay(500);
        int distanceDroite = measureDistance();
        Serial.print("Droite = ");
        Serial.println(distanceDroite);

        // Scan gauche
        servoHead.write(150);
        delay(500);
        int distanceGauche = measureDistance();
        Serial.print("Gauche = ");
        Serial.println(distanceGauche);

        // Retour au centre
        servoHead.write(90);

        delay(300);

        // Décision intelligente
        if (distanceDroite > distanceGauche) {
            droite();
            delay(500);
        } else {
            gauche();
            delay(500);
        }
    }
}

```